C 语言入门

从 Hello, world! 到编译与链接

10 道单选 • 5 道多选 • 5 道填空 • 4 道综合大题

适用范围	标准 C 宿主环境；程序基本结构、stdio.h、main、printf、终止状态与常见构建流程
命题口径	以 WG14 公开委员会草案为规范依据，以 GCC/Clang 官方文档解释工程工具链
难度结构	基础 8 题，中等 11 题，较难 5 题；概念、辨析、代码阅读与工程诊断递进
检索日期	2026年 7 月 25 日

目录

一 图片内容重编码与规范化 **1**
1.1 忠实转写 . . . 1
1.2 推荐的规范版本 . . . 1
1.3 原笔记中需要纠正的两个模型 . . . 1

二 试题 **2**
2.1 单项选择题（共 10 题） . . . 2
2.2 多项选择题（共 5 题） . . . 5
2.3 填空题（共 5 题） . . . 6
2.4 综合大题（共 4 题） . . . 7

三 答案速查 **10**

四 单项选择题极详细解析 **10**

五 多项选择题极详细解析 **14**

六 填空题极详细解析 **16**

七 综合大题极详细解析 **17**
7.1 逐题参考答案、评分点与易错分析 . . . 17

八 知识闭环与自检 **25**

九 检索依据与来源说明 **25**
9.1 命题方法 . . . 25
9.2 主要一手资料 . . . 25
9.3 准确性边界 . . . 26

一 图片内容重编码与规范化

1.1 忠实转写

图片标题为“C语言——第一章：C 语言入门”，正文以 `Hello, world!` 为例。可辨识的核心代码为：

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!");
    return 0;
}
```

图片旁注的含义可重编码为：

- `#include <stdio.h>`：预处理；
- `int main()`：主入口名字；
- 花括号内部：业务代码；
- `return 0;`：结束，且为正常结束；
- 流程：编写→ 编译（转变为 0 与 1）→ 链接（与 `<stdio.h>` 等连接）→ 运行。

1.2 推荐的规范版本

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

为什么这样改？

- `int main(void)` 跨常用 C 版本都能清楚表达“无参数”。图片中的 `int main()` 作为函数定义并不应被草率判为语法错误，但 `(void)` 的教学含义更明确。
- 原图没有 `\n`，所以不会自动换行，成功时 `printf` 返回13。规范版增加换行，成功时返回14。
- `return 0;` 表示向宿主环境报告成功终止；它不会把数字 0 输出到屏幕。

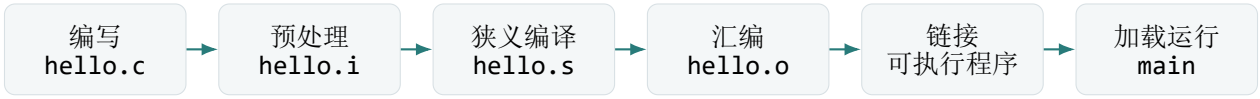
1.3 原笔记中需要纠正的两个模型

易错提醒

误区一：“链接时与 `stdio.h` 连接”。 `#include <stdio.h>` 在预处理阶段起作用，使 `printf` 等声明、宏和类型进入后续翻译过程。链接器处理的是目标文件与库组件，不是头文件。若目标代码保留了对 `printf` 的外部引用，该引用通常在后续链接或装载中由 C 实现提供的库满足。

误区二：“编译就是转成 0 和 1”。 这只能作为极粗略比喻。更有解释力的模型是：源代码先预

处理，再经过狭义编译与汇编形成含机器代码、符号和重定位信息的目标文件，最后由链接器形成程序映像。真实实现可以合并阶段，中间文件不一定真的落盘。



口径说明。本文默认讨论标准 C 的宿主环境。ISO C 的“翻译阶段”是语言规范的概念模型；`gcc -E`、`-S`、`-c` 以及 `.i/.s/.o` 是 GCC/Clang 一类常见工具链的工程视角。二者相关，但不能机械地视为完全相同。

二　试题

单项选择题	10题，每题 2 分	20 分
多项选择题	5题，每题 3 分；少选、多选均不得分	15 分
填空题	5题，每题 2 分	10 分
综合大题	4题，依次 12、13、14、16 分	55 分
合计	24题	**100 分**

2.1　单项选择题（共 10 题）

第 1 题　C 源文件识别 容易 考点：源文件扩展名

在 GCC、Clang 等常见工具链的默认文件名约定中，下列哪个文件通常会被识别为“需要先预处理的 C 源文件”？

A. `hello.h`
B. `hello.c`
C. `hello.i`
D. `hello.o`

第 2 题　#include 的作用阶段 中等 考点：预处理与链接

对指令

```
#include <stdio.h>
```

最准确的描述是：

A. 程序运行时立即把整个标准输入输出库加载到内存
B. 链接器立即查找并复制 `printf` 的机器代码
C. 预处理阶段按实现规定查找标准头，并使该头提供的声明、宏等进入后续翻译过程

D. 把字符串 `stdio.h` 输出到标准输出

第 3 题　标准 main 形式 容易 考点：宿主环境与入口函数

在标准 C 的宿主环境中，下列哪一个是标准明确列出的、无命令行参数的 `main` 定义形式？

A.

```
int main(void)
{
    return 0;
}
```

```
void main(void)
{
}
```

B.

```
int Main(void)
{
    return 0;
}
```

```
int main(int argc)
{
    return 0;
}
```

第 4 题　printf 所属标准头 容易 考点：函数声明

若要通过标准头取得 `printf` 的函数声明，应包含：

A. `<stdlib.h>`

B. `<string.h>`

C. `<math.h>`

D. `<stdio.h>`

第 5 题　printf 返回值与转义字符 中等 考点：字符计数

假设输出成功，执行

```
int n = printf("A\tB\n");
```

后，`n` 的值为：

A. 3

B. 4

C. 5

D. 取决于终端把制表符显示成几列，无法确定

第 6 题　输出百分号 中等 考点：格式字符串

执行

```
printf("rate=100%%\n");
```

其输出效果是：

A. 输出 `rate=100%%`，然后换行

B. 编译错误，因为字符串中不能连续出现两个`%`

C. 输出 `rate=100%`，然后换行

D. 必须再提供一个实参，否则无法输出`%`

第 7 题　return 0 的含义 中等 考点：终止状态

在宿主环境中，从 `main` 执行

```
return 0;
```

其核心含义是：

A. 结束 `main` 的初始调用，并向宿主环境报告成功终止状态

B. 把程序中的全部变量都重新设置为零

C. 从程序开头重新运行一次

D. 清空已经写入标准输出的全部内容

第 8 题　常见构建流程 容易 考点：阶段顺序

在常见的 C 工具链概念模型中，顺序正确的是：

A. 编写源文件→ 链接→ 预处理→ 编译→ 汇编→ 运行

B. 编写源文件→ 预处理→ 汇编→ 编译→ 链接→ 运行

C. 编写源文件→ 编译→ 预处理→ 链接→ 汇编→ 运行

D. 编写源文件→ 预处理→ 编译→ 汇编→ 链接→ 运行

第 9 题　编译错误判断 容易 考点：语法错误

考虑程序：

```
int main(void)
{
    return 0
}
```

缺少分号通常会首先导致哪一阶段报告错误？

A. 预处理阶段

B. 编译阶段的语法分析

C. 汇编阶段

D. 链接阶段

第 10 题　链接错误判断 中等 考点：声明与定义

考虑程序：

```
int helper(void);

int main(void)
{
    return helper();
}
```

假设任何源文件、目标文件或库中都没有 `helper` 的定义。通常会在哪一阶段失败？

A. 预处理阶段

B. 编译阶段，因为函数不能只有声明

C. 链接阶段，因为外部引用无法解析

D. 运行阶段，调用 `helper` 时才会发现

2.2　多项选择题（共 5 题）

每题至少有两个正确选项。请选出所有正确项；少选、多选均不得分。

第 11 题　识别预处理指令 容易 考点：预处理指令与普通语句

下列哪些属于 C 预处理指令？

A. `#include <stdio.h>`

B. `#define LIMIT 100`

C. `#if LIMIT > 0`

D. `printf("hello");`

E. `return 0;`

第 12 题　stdio.h 与 printf 中等 考点：声明、输出流与返回值

关于 `<stdio.h>` 和 `printf`，下列说法正确的有：

A. `<stdio.h>` 提供 `printf` 的标准声明

B. `printf` 的输出目标相当于把 `stdout` 作为 `fprintf` 的流参数

C. 输出成功时，`printf` 返回传送的字符数

D. `printf` 成功时一定返回 `0`

E. 仅仅包含 `<stdio.h>` 并不会自动执行一次输出

第 13 题　main 与程序终止 较难 考点：标准形式与隐式返回

在标准 C 宿主环境中，下列说法正确的有：

A. `int main(void)` 是标准明确列出的形式

B. `int main(int argc, char *argv[])` 是标准明确列出的形式

C. `void main(void)` 是标准规定的可移植形式

D. 执行到返回类型与 `int` 兼容的 `main` 末尾右花括号，终止状态等同于返回 `0`

E. `argc` 必须严格大于 `0`

第 14 题　字符串中的特殊写法 中等 考点：转义序列与格式解释

下列说法正确的有：

A. `\n` 在字符串字面量中表示换行字符

B. `\t` 在字符串字面量中表示水平制表字符

C. `\\` 在字符串字面量中表示一个反斜杠字符

D. `%%` 是 C 词法层面的反斜杠转义序列

E. `\"` 可在字符串字面量中表示双引号字符

第 15 题　错误所属阶段 较难 考点：错误分层

按常见 C 工具链的通常行为，下列分类正确的有：

A. 活动代码中包含一个确定不存在且无法找到的头文件，通常在预处理阶段失败

B. 普通语句末尾缺少必要的分号，通常在编译阶段报告语法错误

C. 函数已有声明且被调用，但程序中没有其定义，通常在链接阶段报告未解析符号

D. 两个目标文件都为同一个外部函数提供普通的强定义，通常在链接阶段报告重复定义

E. 程序成功启动后执行整数除以零，属于链接错误

2.3　填空题（共 5 题）

第 16 题　标准术语 较难 考点：预处理翻译单元

一个源文件连同它通过 `#include` 包含的所有头和源文件，在标准术语中合称为“_____”；完成预处理后，则称为“翻译单元”。

第 17 题　main 的返回类型 容易 考点：标准主函数形式

补全标准的无参数主函数定义：

```
__________ main(void)
{
    return 0;
}
```

第 18 题　printf 的返回值 中等 考点：实际字符数

假设输出成功：

```
int n = printf("X\\Y\n");
```

执行后，n = _____。

第 19 题　构建阶段 容易 考点：工具链流程

补全常见工具链流程：

预处理 ⟶ 编译 ⟶ _____ ⟶ 链接

第 20 题　错误类型 中等 考点：未解析外部符号

某函数已有正确声明并在程序中被调用，各源文件均能生成目标文件，但任何目标文件或库都没有提供该函数的定义。最终构建失败属于_____ 错误。

2.4　综合大题（共 4 题）

第 21 题　程序阅读与纠错 中等 **（12 分）** 考点：语法、整数除法与格式匹配

阅读下面程序：

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 4
    printf("sum=%d\n", a + b);
    printf("average=%.2f\n", (a + b) / 2);
    Return 0;
}
```

1. 找出程序中的所有问题，分别说明其性质及可能后果。（4分）
2. 在保留原有功能的前提下，将程序改为符合标准 C 的正确程序。程序应输出两个整数的和与算术平均数，平均数保留两位小数。（4 分）
3. 写出改正后程序的精确输出。（2分）
4. 说明 `#include <stdio.h>` 和 `return 0;` 各自的作用。（2 分）

第 22 题　从源程序到运行结果 中等 **（13 分）** 考点：完整工具链与错误定位

有如下源文件 `flow.c`：

```c
#include <stdio.h>

#define MESSAGE "Hello, C!"

int main(void)
{
    printf("%s\n", MESSAGE);
    return 0;
}
```

1. 将“汇编、运行、预处理、链接、编译”排成正确顺序，并写出各主要环节的典型产物。（4分）
2. 判断下列说法是否正确并说明理由。（5分）
   A. 预处理发生在编译之前，会处理 `#include` 和 `#define`。
   B. 处理 `#include <stdio.h>` 时，预处理器会把 `printf` 的机器指令复制到源程序中。
   C. 若 `return 0;` 前一条语句漏写分号，这通常属于编译阶段发现的问题。
   D. 某函数已有正确声明但在整个程序中没有定义时，调用该函数一定会在编译单个源文件时失败。
   E. 目标文件通常还不能直接作为最终程序运行，需要经过链接。
3. 指出以下问题通常最早在哪一环节暴露：少写右花括号；已声明的 `greet` 没有可链接定义；程序能运行但公式写错。（3 分）
4. 写出程序的精确输出，并说明 `return 0;` 的含义。（1 分）

第 23 题　带输入检查的简易计算器 较难　（14 分）考点：标准输入、格式匹配与零除防护

编写一个标准 C 程序，完成以下任务：

1. 从标准输入读取两个整数a, b。测试数据保证：成功读入时二者均在$[-100, 100]$ 内，无须另做范围校验。
2. 若不能成功读入两个整数，只输出 `Input error` 并换行，然后以失败状态结束。
3. 输入有效时依次输出$a+b$、$a-b$、$a\times b$。
4. 若$b=0$，最后输出 `quotient=undefined` 并换行；否则输出a/b 的实数结果并保留两位小数。
5. 不输出输入提示语，以便自动测试。

输出格式必须严格如下：

```
sum=...
difference=...
product=...
quotient=...
```

1. 写出完整程序。（8分）
2. 分别写出输入 `8 3`、`8 0` 和 `8 x` 时的精确输出。（3 分）
3. 解释为什么不能把整数表达式 `a / b` 直接传给 `printf` 的`%.2f`。（2 分）
4. 解释为什么必须在执行除法之前判断b 是否为 0。（1 分）

第 24 题　多文件编译与链接诊断 较难　（16 分）考点：声明、定义、目标文件与头文件保护

某程序由以下三个文件组成。

calc.h

```
#ifndef CALC_H
#define CALC_H

int add(int left, int right);

#endif
```

calc.c

```
#include "calc.h"

int add(int left, int right)
{
    return left + right;
}
```

main.c

```
#include <stdio.h>
#include "calc.h"

int main(void)
{
    printf("result=%d\n", add(5, 7));
    return 0;
}
```

现执行：

```
cc -std=c11 -Wall -Wextra -pedantic -c main.c -o main.o
cc main.o -o app
```

第一条命令成功，第二条命令却报告与 `add` 有关的“未定义引用”或“无法解析的外部符号”。

1. 为什么 `main.c` 能够编译成功，而链接失败？（4 分）
2. 写出分别编译、链接并运行该程序的完整命令；再写出一种直接由源文件生成可执行程序的命令。（4分）
3. 诊断以下两种修改最早造成编译错误还是链接错误，并说明理由。（4分）
   - 把 `calc.c` 中的定义名改为 `sum`，其余文件不变；
   - 把 `calc.h` 中声明改为 `double add(double,double);`，而 `calc.c` 保持原来的 `int add(int,int)` 定义。
4. 说明头文件中为什么通常只放函数声明，并解释头文件保护宏的作用。（3分）
5. 写出正确程序的精确输出与正常结束状态。（1分）

说明： `cc`、`-std=c11`、`-Wall` 等是 GCC/Clang 一类工具链的常见命令和选项，不是 ISO C 标准规定的命令。

三 答案速查

客观题答案

题号	答案	题号	答案	题号	答案	题号	答案
1	B	**6**	C	**11**	A、B、C	**16**	预处理翻译单元
2	C	**7**	A	**12**	A、B、C、E	**17**	`int`
3	A	**8**	D	**13**	A、B、D	**18**	4
4	D	**9**	B	**14**	A、B、C、E	**19**	汇编
5	B	**10**	C	**15**	A、B、C、D	**20**	链接

综合题核心结果。 第 21 题输出为 `sum=11` 与 `average=5.50`；第 22 题流程为“预处理—编译—汇编—链接—运行”，输出 `Hello, C!`；第 23 题须检查 `scanf` 返回值、零除并在除法前转为 `double`；第 24 题链接命令遗漏 `calc.o`，正确输出为 `result=12`。

四 单项选择题极详细解析

第 1 题 C 源文件识别答案：B

核心推理。 在 GCC、Clang 等常见工具链的默认约定中，后缀 `.c` 表示尚未预处理的 C 源代码。驱动程序看到 `hello.c` 时，通常会从预处理开始；如果没有指定停止选项，还会继续完成编译、汇编和链接。

逐项辨析。

✗ **A `hello.h`**：通常表示头文件，往往通过 `#include` 进入某个源文件的预处理翻译单元。GCC 直接接收 `.h` 时还可能按预编译头处理。

✓ **B `hello.c`**：通常表示需要预处理的 C 源文件。

✗ **C `hello.i`**：GCC 约定中通常表示已经预处理过的 C 文本，驱动程序一般不再执行普通预处理。

✗ **D `hello.o`**：通常是目标文件，是汇编后的产物及链接器的输入之一，不是 C 源文本。

边界意识： ISO C 标准本身不要求操作系统文件必须使用 `.c` 后缀。题干已经把语境限定为 GCC/Clang 等常见工具链，因此这里考查的是工具约定，而不是语言语法。

第 2 题 `#include` 的作用阶段答案：C

核心推理。 `#include` 是预处理指令。标准的概念模型在翻译阶段 4 执行预处理指令；尖括号形式按实现规定的位置搜索由 `stdio.h` 标识的标准头，并使其中的声明、宏和类型等参与后续翻译。

逐项辨析。

✗ **A**：包含头发生在翻译期间，不是程序运行时的动态加载动作；更不等于把“整个 I/O 库”立即放进内存。

✗ **B**：此时还没有进入链接阶段。头提供接口信息；`printf` 的实现不是由预处理器复制进源文件的机器指令。

✓ **C**：准确指出了处理阶段、搜索方式的实现相关性，以及标准头对后续翻译的作用。

✗ **D**：尖括号内的是头名称，不是输出字符串。

易错提醒

“包含头”与“链接库”是两件不同的事。前者主要让编译器看到正确声明；后者把目标文件中的外部引用与目标文件或库中的定义相匹配。链接器不链接 `stdio.h`。

第 3 题　标准 main 形式答案：A

核心推理。　标准 C 在宿主环境中明确列出：

```
int main(void)
```

以及带 `argc/argv` 的两参数形式，或类型上等价的写法。选项 A 返回 `int`，函数名大小写正确，且 `(void)` 明确表示没有参数。

逐项辨析。

✓ **A**：标准直接列出的无参数形式。

✗ **B**：`void main(void)` 可能被某些实现当作扩展接受，但不是标准宿主环境中的可移植形式。

✗ **C**：C 标识符区分大小写，`Main` 与 `main` 是不同名称。

✗ **D**：标准明确列出的带参数形式包含 `argc` 和 `argv` 两个参数；单独一个 `int` 参数不是所要求的可移植形式。

版本说明：本题没有把 `int main()` 设计成干扰项。图片中的这种函数定义不应被简单判错；教学中使用 `int main(void)`，是因为“无参数”的意图跨常用版本最清楚。

第 4 题　printf 所属标准头答案：D

核心推理。　`printf` 是标准格式化输出函数，原型由 `<stdio.h>` 提供。其核心形式为：

```
int printf(const char * restrict format, ...);
```

逐项辨析。

✗ **A `<stdlib.h>`**：主要提供通用工具，如内存分配、数值转换、程序终止与搜索排序等。

✗ **B `<string.h>`**：主要提供字符串和内存块操作。

✗ **C `<math.h>`**：主要提供数学函数。

✓ **D `<stdio.h>`**：标准输入输出头，提供 `printf` 的声明。

易错提醒

头文件中的函数声明不是函数实现本体。声明回答“怎样正确调用”；实现回答“函数体真正做什么”。这一区分会在第 10 题和第 24 题再次出现。

第 5 题　printf 返回值与转义字符答案：B，即 4

逐字符计数。　字符串`"A\tB\n"` 在执行时包含：

$$\underbrace{\texttt{A}}_{1}\quad\underbrace{\texttt{\textbackslash t}}_{1}\quad\underbrace{\texttt{B}}_{1}\quad\underbrace{\texttt{\textbackslash n}}_{1}$$

所以输出成功时：

$$n = 1 + 1 + 1 + 1 = 4.$$

逐项辨析。

✗ **A（3）**：通常是漏算了换行字符。

✓ **B（4）**：四个实际传送的字符。

✗ **C（5）**：通常把源代码中的反斜杠和字母 `t` 当成两个输出字符；实际转义序列表示一个字符。

✗ **D**：制表符在屏幕上跨越多少列可以因终端而异，但 `printf` 统计的是传送的字符数，不是视觉列数。

题干已假设输出成功。若发生输出或编码错误，标准允许 `printf` 返回负值。

第 6 题　输出百分号 答案：C

核心推理。 在 `printf` 的格式字符串里，`%%` 表示输出一个普通百分号，而且不读取附加实参。因此输出为：

```
rate=100%
```

随后由 `\n` 换行。

逐项辨析。

✗ **A**：两个百分号不是原样输出；它们共同形成一个“输出字面百分号”的格式说明。

✗ **B**：`%%` 是标准允许的格式写法。

✓ **C**：输出一个`%`，再换行。

✗ **D**：与`%d`、`%s` 不同，`%%` 不消耗实参。

返回值延伸。 成功时，`rate=` 有 5 个字符，`100` 有 3 个，百分号 1 个，换行 1 个，所以该调用返回$5 + 3 + 1 + 1 = 10$。

易错提醒

`\n` 是 C 字符串字面量的反斜杠转义；`%%` 是 `printf` 在运行时解释格式字符串时识别的写法。两者处在不同处理层次。

第 7 题　return 0 的含义 答案：A

核心推理。 从 `main` 的初始调用执行 `return 0;`，会结束 `main`，把 0 作为程序终止状态交给宿主环境。标准规定，从返回类型与 `int` 兼容的 `main` 返回，效果相当于以该值调用 `exit`；值 0 表示成功终止。

逐项辨析。

✓ **A**：同时说明了控制流结束和成功状态报告。

✗ **B**：它不会逐一清零变量或内存。

✗ **C**：`return` 离开函数；从初始 `main` 返回会终止程序，不是重启。

✗ **D**：正常终止不会撤销已经产生的输出；反而会按规则处理缓冲流的刷新与关闭。

在现代标准 C 的宿主环境中，到达 `main` 末尾右花括号也等价于返回 0。显式写出 `return 0;` 仍很适合初学阶段，因为意图清楚。

第 8 题　常见构建流程答案：D

正确顺序。

编写 → 预处理 → 编译 → 汇编 → 链接 → 运行

- **预处理**：执行 `#include`、`#define`、条件包含等；
- **编译**：分析预处理后的 C 程序并生成汇编表示或等价中间结果；
- **汇编**：产生目标文件；
- **链接**：合并目标文件与所需库，解析外部符号并形成程序映像；
- **运行**：执行环境加载并启动程序。

逐项辨析。 A把链接提前；B 把汇编放在编译之前；C 把预处理放在编译之后且把链接放在汇编之前；只有 D 的依赖顺序正确。

实现可以把多个阶段合并在一个进程中，不要求所有`.i/.s/.o` 文件都真实存在。概念顺序不等于必须逐步落盘。

第 9 题　编译错误判断答案：B

核心推理。 `return 0` 后缺少分号，违反 C 语法。预处理结束后，编译器前端在解析返回语句时发现缺少必要的`;`，因此通常在编译阶段给出诊断。

逐项辨析。

✗ **A**：预处理器处理预处理指令与预处理记号，不负责判断普通返回语句是否缺分号。

✓ **B**：这是典型语法错误。

✗ **C**：源程序尚未成功编译，正常情况下不会形成可供汇编器使用的有效结果。

✗ **D**：链接器处理目标文件、符号与重定位，不解析 C 源代码中的分号。

易错提醒

编译器驱动程序可能统一显示来自不同阶段的错误，但“由同一个命令显示”不等于“属于同一阶段”。判断时应看错误的本质。

第 10 题　链接错误判断答案：C

声明为什么足以通过编译。

```
int helper(void);
```

告诉编译器：`helper` 是一个无参数、返回 `int` 的函数。因此编译器能够检查调用方式，并在目标文件中留下对外部符号 `helper` 的引用。

为什么仍会链接失败。 声明不提供函数体。链接器必须在其他目标文件或库中找到 `helper` 的定义。题干明确说明所有链接输入都没有定义，因此产生 `undefined reference` 或 `unresolved external symbol` 一类错误。

逐项辨析。

✗ **A**：没有导致头搜索或宏处理失败的内容。

✗ **B**：C 允许调用者翻译单元只看到声明，定义放在另一个翻译单元或库中。

✓ **C**：链接器无法满足外部引用。

✗ **D**：普通静态构建通常无法形成完整可执行程序，不会等到运行才发现。

声明让调用“可编译” ≠ 定义让符号“可链接”

五　多项选择题极详细解析

第 11 题　识别预处理指令答案：A、B、C

判断标准。　预处理指令在允许的位置以 # 预处理记号开头，并在预处理阶段被识别。

✓ **A** `#include`：源文件包含指令。

✓ **B** `#define`：宏定义指令。

✓ **C** `#if`：条件包含指令的一部分。

✗ **D** `printf("hello");`：函数调用表达式构成的普通 C 语句。

✗ **E** `return 0;`：返回语句，不是预处理指令。

> # 前可以出现允许的空白，所以“# 必须是物理行绝对第一个字符”并不严谨。关键是它在翻译阶段 4 开始时出现在能够开始预处理指令的位置。

第 12 题　stdio.h 与 printf答案：A、B、C、E

✓ **A**：`<stdio.h>` 提供 `printf` 的标准声明，其返回类型为 `int`。

✓ **B**：标准规定 `printf` 等价于在参数中插入 `stdout` 后调用 `fprintf`，所以普通 `printf` 面向标准输出流。

✓ **C**：成功时返回传送的字符数；输出或编码错误时返回负值。

✗ **D**：成功返回值不是固定 0。例如 `printf("abc")` 成功时返回 3。

✓ **E**：包含头只提供接口信息等；没有函数调用，就不会凭空产生运行期输出。

易错提醒

不要混淆 `printf` 与 `scanf` 的返回值：前者通常返回输出字符数；后者通常返回成功赋值的输入项数。

第 13 题　main 与程序终止答案：A、B、D

✓ **A**：`int main(void)` 是标准直接列出的无参数形式。

✓ **B**：`int main(int argc, char *argv[])` 是标准列出的两参数形式；`char **argv` 等类型上等价的写法也可以。

✗ **C**：标准宿主环境要求 `main` 返回 `int`；`void main` 属于某些实现的扩展，不可移植。

✓ **D**：若 `main` 返回类型与 `int` 兼容，到达末尾右花括号等价于返回 0。

✗ **E**：标准要求 `argc` 非负，即`argc` ≥ 0，不是必须严格大于 0。

特殊规则不可乱推广：“到达末尾等价于返回 0”是 `main` 的特殊规则。普通返回非 `void` 的函数如果其返回值会被使用，却沿着未返回值的路径离开，不能照搬这条结论。

第 14 题　字符串中的特殊写法答案：A、B、C、E

✓ **A**：`\n` 表示换行字符。
✓ **B**：`\t` 表示水平制表字符。
✓ **C**：`\\` 在字符串的值中产生一个反斜杠。
✗ **D**：`%%` 没有反斜杠，不是 C 词法层面的转义序列；它由 `printf` 解释格式字符串时识别。
✓ **E**：`\"` 可在字符串字面量内部表示双引号字符。

三个处理层次。

1. C 源代码词法处理识别 `\n`、`\t`、`\\`、`\"`；
2. 字符串形成后，`printf` 再解释其中以`%` 开始的转换说明；
3. 输出设备最终决定制表和换行的视觉效果。

第 15 题　错误所属阶段答案：A、B、C、D

✓ **A**：活动的 `#include` 若确定无法找到所指头，通常在预处理时失败。
✓ **B**：缺少分号属于 C 语法问题，由编译器前端报告。
✓ **C**：正确声明足以让调用者翻译单元编译；没有定义则通常在链接时暴露。
✓ **D**：两个目标文件为同一外部函数各提供普通强定义，链接器通常报告多重定义或重复符号。
✗ **E**：程序已经启动，说明链接早已结束。整数除以零属于运行期间的未定义行为，不是链接错误。

阶段	典型问题
预处理	头文件找不到、预处理条件或指令问题
编译	缺分号、括号不配对、类型与语言约束问题
链接	未定义引用、重复定义、目标文件或库遗漏
运行	崩溃、未定义行为、输入异常、逻辑输出错误

六　填空题极详细解析

第 16 题　标准术语答案：预处理翻译单元

标准把“一个源文件及其通过 `#include` 纳入的全部头和源文件”称为**预处理翻译单元**（preprocessing translation unit）。

$$\boxed{\text{源文件} + \text{递归包含的头或源文件} = \text{预处理翻译单元}}$$

$$\boxed{\text{完成预处理} \Rightarrow \text{翻译单元}}$$

为什么其他答案不合适。

- “源文件”没有体现递归包含的内容；
- “目标文件”是后续工具链产物，不是这一定义；
- “可执行文件”要到链接以后才形成。

第 17 题　`main` 的返回类型答案：`int`

完整形式为：

```
int main(void)
{
    return 0;
}
```

`int` 是标准宿主环境中 `main` 的返回类型，返回值用于表示程序终止状态。不能填 `void`；现代标准 C 也不能依赖旧式“省略返回类型便隐含为 `int`”。

第 18 题　`printf` 的返回值答案：4

源字符串是：

```
"X\\Y\n"
```

逐个解释：

$$\underbrace{\texttt{X}}_{1} + \underbrace{\texttt{\textbackslash\textbackslash}}_{\text{一个反斜杠}} + \underbrace{\texttt{Y}}_{1} + \underbrace{\texttt{\textbackslash n}}_{\text{一个换行}} = 4.$$

输出可抽象表示为：

```
X\Y
```

随后换行。计数以实际传送字符为准，不按源代码为了书写转义而出现的字符数量计算。

第 19 题　构建阶段答案：汇编

完整流程为：

$$\text{预处理} \rightarrow \text{编译} \rightarrow \boxed{\text{汇编}} \rightarrow \text{链接}.$$

狭义编译通常产生汇编表示或等价中间结果；汇编阶段产生目标文件；链接阶段再组合目标文件与所需库。实现可以融合这些工作，所以用户不一定看见所有中间文件。

第 20 题　错误类型答案：链接

推理链如下：

1. 函数声明让编译器知道调用接口；
2. 各源文件已经生成目标文件，说明预处理、编译与汇编已完成；
3. 调用处在目标文件中留下外部引用；
4. 所有链接输入都没有相应定义；
5. 链接器无法解析引用，产生链接错误。

典型诊断文字包括：

```
undefined reference to ...
unresolved external symbol ...
```

七　综合大题极详细解析

7.1　逐题参考答案、评分点与易错分析

第 21 题　程序阅读与纠错12 分参考答案

1. 问题定位

原程序至少有四个必须处理的问题。

问题一：声明末尾缺少分号。

```
int a = 7, b = 4
```

应为：

```
int a = 7, b = 4;
```

分号结束这条声明。缺少它会使后续 `printf` 无法与声明正确分隔，编译器必须给出语法诊断。

问题二：关键字大小写错误。　`Return` 不是 C 关键字。C 区分大小写，应改为：

```
return 0;
```

词法分析会把 `Return` 当作普通标识符；`Return 0;` 不构成合法返回语句。

问题三：发生整数除法。

```
(a + b) / 2
```

的所有操作数都是 `int`，因此执行整数除法：

$$a+b=11, \qquad 11/2=5.$$

小数部分在除法当时已经被舍去，并不会得到5.5。

问题四：格式说明与实参类型不匹配。　`%.2f` 要求对应实参是 `double`，但 `(a+b)/2` 的类型为 `int`。由于 `printf` 是可变参数函数，它会按照格式字符串解释后续实参；类型不匹配会导致未定义行为，不能把它理解为“自动把 5 变成 5.00”。

易错提醒

原程序先有编译错误，因此没有标准规定的运行输出。即使只修复分号和大小写而保留错误的`%.2f` 调用，程序的执行行为仍未定义；不能以某台机器的一次偶然结果作为答案。

2. 符合要求的程序

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 4;

    printf("sum=%d\n", a + b);
    printf("average=%.2f\n", (a + b) / 2.0);

    return 0;
}
```

常量 `2.0` 的类型为 `double`。除法前，$a+b$ 的结果会参与浮点运算，所以：

$$\frac{11.0}{2.0} = 5.5.$$

另一种正确写法是：

```c
(double)(a + b) / 2
```

转换发生在除法之前。下式看似相近，结果却仍是5.0：

```c
(double)((a + b) / 2)
```

原因是括号内部先完成整数除法$11/2 = 5$，之后才转换，已经丢失的小数部分无法恢复。

3. 精确输出

```
sum=11
average=5.50
```

`%.2f` 要求小数点后显示两位，因此5.5 显示为 `5.50`。每行末尾的 `\n` 产生换行。

4. 两个关键结构的作用

- `#include <stdio.h>` 是预处理指令，使该翻译单元取得标准输入输出接口信息，包括 `printf` 的声明。它不是“把 `printf` 的机器代码粘贴进源文件”，也不是链接头文件。
- `return 0;` 从 `main` 返回 0，表示正常终止并向宿主环境报告成功状态；它不会自动显示数字 0。

评分参考

评分点	分值
找出缺分号与 `Return` 大小写错误	2
同时指出整数除法和`%.2f`/`int` 类型不匹配	2
给出可编译且计算正确的完整程序	4
两行输出完全正确	2
正确解释标准头与终止状态	2

常见失分点

- 只把`%.2f`改成`%d`：类型虽匹配，却没有实现“两位小数”。
- 写成`(double)((a+b)/2)`：转换发生太晚。
- 认为`return 0;`会在屏幕上输出 0。
- 认为原程序只是“输出数字不准确”，忽略它首先不能正确翻译。

第 22 题　从源程序到运行结果13 分参考答案

1. 正确流程与典型产物

源文件 → 预处理 → 编译 → 汇编 → 链接 → 运行

环节	典型产物	本题中的主要作用
源文件	`flow.c`	保存人可读的 C 源代码
预处理	`flow.i`	处理 `#include`，把宏 `MESSAGE` 展开为字符串
编译	`flow.s`	进行词法、语法、语义分析与代码生成
汇编	`flow.o`	形成含机器代码、符号与重定位信息的目标文件
链接	`flow` 或 `flow.exe`	解析外部引用并形成可执行程序
运行	进程及其输出	执行启动过程并调用 `main`

GCC/Clang 兼容工具链可用以下命令观察典型阶段：

```
cc -std=c11 -E flow.c -o flow.i
cc -std=c11 -S flow.i -o flow.s
cc -c flow.s -o flow.o
cc flow.o -o flow
```

也可以直接让驱动程序组织全部步骤：

```
cc -std=c11 -Wall -Wextra -pedantic flow.c -o flow
```

不要机械化： ISO C 不要求必须使用这些文件扩展名，也不要求每一步都写出中间文件。GCC/Clang 还可能使用集成汇编器，把阶段融合起来。这里是便于学习的工程模型。

2. 五项判断

✓ **A 正确。** `#include` 与 `#define` 都在预处理阶段处理；宏 `MESSAGE` 会在正式分析 C 表达式前展开。

✗ **B 错误。**标准头提供声明等接口信息，不是 `printf` 的机器指令本体。函数实现通常由 C 实现的库提供，并在链接或装载过程中满足引用。

✓ **C 正确。**普通语句漏写分号破坏 C 语法，通常由编译器前端发现。

✗ **D 错误。**正确声明足以让调用者源文件生成目标文件；若最终没有定义，通常到链接时才报告未定义符号。

✓ **E 正确。**目标文件可能仍含未解析引用和重定位信息，通常还不能作为最终程序直接运行。

3. 三类问题最早暴露的环节

- 少写右花括号：**编译阶段**的语法问题；
- `greet` 已声明但没有可链接定义：**链接阶段**的未定义引用；
- 程序能生成并运行，但公式写错：**运行/测试阶段**表现出的逻辑错误。

$$\text{语法正确} \not\Rightarrow \text{链接成功} \not\Rightarrow \text{逻辑正确}.$$

4. 输出与终止状态

宏展开后，关键调用在效果上相当于：

```
printf("%s\n", "Hello, C!");
```

精确输出为：

```
Hello, C!
```

末尾有换行。`return 0;` 表示成功终止，不会再额外输出字符。

评分参考

评分点	分值
五个环节顺序正确，并说明典型产物	4
A–E 判断与理由完整	5
三类问题的阶段定位正确	3
输出与返回状态正确	1

常见失分点

- 把头文件当作库文件或机器代码本体；
- 认为生成 `.o` 就一定能直接运行；
- 认为所有函数问题都在编译阶段发现，混淆声明与定义；
- 把 GCC/Clang 命令行选项误认为 ISO C 语法的一部分。

第 23 题　带输入检查的简易计算器14 分参考答案

1. 完整程序

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
```

```c
    int a, b;

    if (scanf("%d%d", &a, &b) != 2) {
        printf("Input error\n");
        return EXIT_FAILURE;
    }

    printf("sum=%d\n", a + b);
    printf("difference=%d\n", a - b);
    printf("product=%d\n", a * b);

    if (b == 0) {
        printf("quotient=undefined\n");
    } else {
        printf("quotient=%.2f\n", (double)a / b);
    }

    return 0;
}
```

为什么要检查 scanf 返回值。 `scanf("%d%d", &a, &b)` 在匹配成功时给两个 `int` 对象赋值，返回成功赋值的输入项数。只有返回 2，才能保证a, b 都由本次输入获得有效值。若直接忽略返回值，未成功赋值的变量可能保留不确定值，后续使用会失去可靠语义。

为什么使用 EXIT_FAILURE。 `EXIT_FAILURE` 由 `<stdlib.h>` 提供，用来向宿主环境报告失败。正常路径返回 0，表示成功。

题设范围为什么重要。 因为$a, b \in [-100, 100]$，所以：

$$|a+b| \le 200, \qquad |a-b| \le 200, \qquad |ab| \le 10000.$$

这些结果落在标准 C 保证的最小 `int` 范围内，因此题设范围下上述和、差、积不会产生有符号整数溢出。

2. 三组精确输出

输入 8 3

```
sum=11
difference=5
product=24
quotient=2.67
```

其中：

$$\frac{8}{3} = 2.6666\ldots$$

在通常默认的“舍入到最近”模式下，`%.2f` 显示两位小数，得到 `2.67`。

输入 8 0

```
sum=8
difference=8
product=0
quotient=undefined
```

程序先判断$b = 0$，所以没有执行危险的除法。

输入 8 x

```
Input error
```

scanf 能把第一个输入转换为整数 8，却无法把 x 按%d 转换为第二个整数，所以返回 1；因为$1 \neq 2$，进入错误分支。

3. 为什么不能直接输出 a / b

如果a, b 都是 int，表达式：

```
a / b
```

执行整数除法。例如$8/3 = 2$，不是$2.666\ldots$。而且该表达式的类型仍为 int，但 printf 的%.2f 要求对应参数为 double：

```
printf("%.2f\n", a / b);  /* wrong */
```

格式与实参真实类型不匹配会产生未定义行为。
正确写法是：

```
(double)a / b
```

先把a 转换为 double，b 也随之参与浮点运算，最后产生 double，与%.2f 匹配。

易错提醒

(double)(a / b) 不等价。它先执行整数除法得到 2，再转换成2.0；转换发生得太晚。

4. 为什么必须先判断零除

整数除数为 0 时执行 a / b 产生未定义行为，可能异常终止，也可能出现其他不可依赖的现象。不能“先除再检查”，因为危险运算已经发生。
正确控制流是：

$$b = 0 \Rightarrow \text{只输出 undefined}, \qquad b \neq 0 \Rightarrow \text{执行浮点除法}.$$

评分参考

评分点	分值
所需头文件、main 结构与变量定义正确	1
scanf 的地址参数及返回值检查正确	3
和、差、积的输出表达式与格式正确	2
零除判断位于除法之前	1
浮点转换和%.2f 匹配正确	1
三组输出完全正确	3
整数除法、格式类型与零除解释正确	3

常见失分点

- 写成 scanf("%d%d", a, b)，遗漏地址运算符；
- 不检查返回值，随后使用未成功赋值的变量；
- 写成 if (b = 0)，把赋值和比较混淆；
- 在检查b 之前已经计算 a/b；
- 输出额外提示语，破坏题目要求的精确输出。

第 24 题　多文件编译与链接诊断16 分参考答案

1. 为什么编译成功而链接失败

`main.c` 包含 `calc.h` 后，能看到：

```
int add(int left, int right);
```

这条声明告诉编译器函数名、参数类型与返回类型，所以编译器能够检查 `add(5, 7)` 的调用并生成 `main.o`。
但是，声明没有函数体。`main.o` 中仍保留一个等待解析的外部引用 `add`。原链接命令：

```
cc main.o -o app
```

没有加入由 `calc.c` 产生的目标文件，因此链接器看不到 `add` 的定义，报告未定义引用。

声明提供接口，允许编译调用
定义提供实体，允许链接解析

2. 正确命令

分别编译：

```
cc -std=c11 -Wall -Wextra -pedantic -c main.c -o main.o
cc -std=c11 -Wall -Wextra -pedantic -c calc.c -o calc.o
```

链接：

```
cc main.o calc.o -o app
```

POSIX 类环境运行：

```
./app
```

Windows 环境通常运行 `app.exe`。
也可以让驱动程序一次处理两个源文件：

```
cc -std=c11 -Wall -Wextra -pedantic main.c calc.c -o app
```

一次命令并不表示没有编译和链接，而是驱动程序自动组织各阶段。

3. 两种修改的诊断

修改一：定义改名为 `sum`。 `main.o` 需要符号 `add`，而 `calc.o` 提供的是 `sum`。两个源文件本身通常仍可生成目标文件，但链接器找不到 `add`，所以最早是**链接错误**。

`main.o: needs add`　`calc.o: provides sum`　⇒ unresolved add.

修改二：头文件声明与定义类型冲突。 若头文件变为：

```
double add(double left, double right);
```

而 `calc.c` 包含该头后仍定义：

```
int add(int left, int right)
```

则同一翻译单元先看到一个函数类型，随后又看到不兼容的定义。返回类型和参数类型都冲突，所以编译 `calc.c` 时就必须诊断，最早是**编译错误**，无需等到链接。

这正是实现文件应包含自己头文件的原因之一：编译器可以尽早检查声明与定义是否一致。

4. 头文件与保护宏

头文件通常保存多个翻译单元共享的接口，例如函数声明、类型声明、宏和必要常量。普通外部函数定义通常只放在一个`.c`文件中；若把普通外部函数体直接放进头文件并被多个`.c`文件包含，可能形成多个同名定义，最终导致链接器报告重复定义。
保护宏：

```
#ifndef CALC_H
#define CALC_H
/* declarations */
#endif
```

在同一个翻译单元内的工作过程是：

1. 第一次包含时，`CALC_H`尚未定义，处理头内容并定义它；
2. 再次直接或间接包含时，条件不成立，跳过重复内容。

易错提醒

保护宏防止的是**同一翻译单元内**重复展开，不是“整个程序所有`.c`文件只包含一次”。每个源文件分别预处理，形成各自的翻译单元。

5. 输出与结束状态

$$\mathrm{add}(5,7) = 5 + 7 = 12.$$

精确输出为：

```
result=12
```

末尾有换行，随后`return 0;`报告成功终止状态。

评分参考

评分点	分值
区分声明、定义与未解析外部引用，并指出遗漏`calc.o`	4
两条编译命令、链接命令、运行命令与直接构建命令正确	4
修改一判断为链接错误并说明符号名不一致	2
修改二判断为编译错误并说明函数类型冲突	2
正确说明头文件职责与保护宏	3
输出和成功状态正确	1

常见失分点

- 把`calc.h`当作需要单独编译并参与链接的目标文件；
- 认为有声明就等于已有实现；
- 编译了`calc.c`，却忘记在最终链接命令中加入`calc.o`；

- 只检查函数名，忽略参数类型与返回类型必须一致；
- 省略 `calc.c` 对 `calc.h` 的包含，失去声明–定义一致性的直接检查。

八 知识闭环与自检

必须掌握的结论	精确表述
`#include <stdio.h>`	预处理指令；提供标准输入输出接口信息，不等于链接头文件
`main`	宿主环境启动时调用的函数；标准可移植形式返回 `int`
`printf`	写向 `stdout`；成功返回传送字符数，错误时返回负值
`\n`	一个换行字符；`printf` 不会自动追加换行
`return 0;`	结束 `main` 的初始调用并报告成功终止
预处理	处理包含、宏与条件指令；GCC常用 `-E`
狭义编译	分析并生成汇编表示；GCC常用 `-S` 停止在此后
汇编	形成目标文件；GCC `-c` 完成到目标文件但不链接
链接	合并目标文件和库，解析外部引用，形成程序映像
错误分层	头搜索 → 预处理；语法/类型 → 编译；符号解析 → 链接；逻辑/崩溃 → 运行

一条完整的诊断路径： 先问“源代码是否通过预处理与语法/类型检查”，再问“所有外部符号是否都有定义”，最后问“运行结果是否符合需求”。这三个问题分别对应翻译、链接和测试，不可相互替代。

九 检索依据与来源说明

9.1 命题方法

题目依据图片主题进行网络检索后重新设计，均为原创改编，没有逐字复制网络题库。为保证准确性，规范结论优先采用 WG14 公开委员会草案；工程命令和中间产物采用 GCC、Clang 官方文档；Microsoft 文档只用于交叉观察实现行为与平台差异。访问日期均为 2026 年 7 月 25 日。

9.2 主要一手资料

1. **WG14 N1570：C11 委员会草案**
   https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
   主要使用：§5.1.1.2 翻译阶段；§5.1.2.2.1 `main`；§5.1.2.2.3 程序终止；§6.10.2 头文件包含；§7.21.1 `stdio.h`；§7.21.6.3 `printf`；§7.22.4.4 `exit`。

2. **WG14 N3220：ISO/IEC 9899:2024 后续公开工作草案**
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf
用于核对当前标准框架下的翻译、宿主环境、`main`、标准输入输出与程序终止条款。
3. **GCC 官方：Options Controlling the Kind of Output**
https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html
用于核对预处理、狭义编译、汇编、链接的顺序，以及 `-E`、`-S`、`-c`、`-o` 和常见中间文件。
4. **GCC 官方：Include Syntax**
https://gcc.gnu.org/onlinedocs/cpp/Include-Syntax.html
用于核对尖括号与引号形式的典型用途及头搜索模型。
5. **GCC 官方：Warning Options**
https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
用于说明 `-Wall`、`-Wextra`、`-Wpedantic` 与 `-pedantic-errors` 的诊断口径。
6. **Clang 官方：Assembling a Complete Toolchain**
https://clang.llvm.org/docs/Toolchain.html
用于核对预处理、解析与语义分析、代码生成、汇编、链接的职责，以及集成阶段的现实实现。
7. **Clang 官方命令指南**
https://clang.llvm.org/docs/CommandGuide/clang.html
用于交叉核对 `-E`、`-S`、`-c` 等阶段控制。
8. **Microsoft Learn：main function and program execution**
https://learn.microsoft.com/en-us/cpp/c-language/main-function-and-program-execution?view=msvc-170
用于对比标准入口形式与实现扩展，强调“编译器接受”不等于“ISO C 可移植”。
9. **Microsoft Learn：printf family**
https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/printf-printf-l-wprintf-wprintf-l?view=msvc-170
用于交叉核对 `stdout`、返回值和格式化输出行为。
10. **Microsoft Learn：Linking**
https://learn.microsoft.com/en-us/cpp/build/reference/linking?view=msvc-170
用于对比 Windows 工具链中的`.obj/.exe` 与目标文件链接模型。

9.3 准确性边界

- 本册讨论宿主环境；自由环境的启动函数名称和类型可由实现规定。
- “`.i/.s/.o`”是常见工具链约定，不是 ISO C 强制的文件名。
- 诊断阶段按常见 GCC/Clang 构建理解；具体文字、是否在一次驱动调用中提前终止、链接与装载的实现细节可因平台而异。
- 所有 `printf` 字符计数题都明确假设没有输出或编码错误。
- 题目采用 `int main(void)` 避免版本歧义；没有把图片中的 `int main()` 草率判为必错。

——完——

图片已重编码，24道题目数量复核无误，答案统一置于题目之后。